

Module Name: MySql

Batch: PG-DAC Aug 2025

Q.1) To find the maximum sum of all subarrays of size K:

Given an array of integers of size 'n', Our aim is to calculate the maximum sum of 'k' consecutive elements in the array.

Input : arr[] = {100, 200, 300, 400}, k = 2

Output : 700

Input : arr[] = {1, 4, 2, 10, 23, 3, 1, 0, 20}, k = 4

Output : 39

We get maximum sum by adding subarray {4, 2, 10, 23} of size 4.

Input : arr[] = {2, 3}, k = 3

Output : Invalid

There is no subarray of size 3 as size of whole array is 2.

Q.2) ELIMINATION OF DUPLICATES

DESCRIPTION: Write a function named "remove duplicates" that takes an array of integers in random order and eliminates all the duplicate integers in the array. The function should take two arguments:

(1) An array of integers

(2) An integer that tells the number of cells An array

'a': The integer array of numbers

'n': The number of integers An the array

RETURNS:

The function should not return a value, but if any duplicate integers are eliminated, then array is restructured such that the unique value precedes repeated values.

EXAMPLE:

If input is

int a[11]={58,26,91,26,70,70,91,58,58,58,66}

Revised array:

Q.3) PRIME NUMBER (WORKING CODE ON PC)

DESCRIPTION: Write a program to take an input number from user between the range of 1 and 1000 and return true if the integer is a prime number. Implement the following:

- The user is allowed to enter numbers only in the allowed range. If the user enters number out of range, ask him to re-enter
- Once a number is entered within range, your program will compute whether prime or not and display and then program will continue to ask him to put another number. This will go on until user presses '0'
- User can press '0' to exit the program

Q.4) Count occurrence of letter 'A' with Big O(1) from given set of data

string[] arr = { "A", "B", "C", "D", "E", "F", "F", "G", "A" };

A occurred 2 time

F occurred 2 time

Q.5) Custom Fibonacci Sequence

Given n (0-based indexing), return the n -th Fibonacci number where $F(0) = 1$, $F(1) = 2$, and $F(n) = F(n-1) + F(n-2)$.

Examples

Example 1

Input:

$n = 3$

Output:

5

Explanation:

- Compute intervals step by step: $\text{interval}[0] = 1$, $\text{interval}[1] = 2$. Then $\text{interval}[2] = 1 + 2 = 3$, $\text{interval}[3] = 2 + 3 = 5$. So for $n = 3$, the function returns 5.

Example 2

Input:

n = 10

Output:

144

Explanation:

- List the first 11 intervals: [1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144].
- Each interval is the sum of the two previous ones. The 10th index (0-based) is 144.

Input Format

- The first and only line of input contains a single integer, n.

Constraints

- $0 \leq n \leq 92$
- n is an integer

Output Format

- Return a single integer: the computed auto-save interval in minutes for the given input n.

Sample Input 0

0

Sample Output 0

1

Sample Input 1

1

Sample Output 1

2

Q.6) Accept a number from user and print it's binary number.

Q.7) Given an array of integers **arr[]**, the task is to move all the **zeros** to the **end** of the array while maintaining the relative order of all **non-zero** elements.

Examples:

Input: *arr[] = [1, 2, 0, 4, 3, 0, 5, 0]*

Output: *arr[] = [1, 2, 4, 3, 5, 0, 0, 0]*

Explanation: *There are three 0s that are moved to the end.*

Input: $arr[] = [10, 20, 30]$

Output: $arr[] = [10, 20, 30]$

Explanation: No change in array as there are no 0s.

Input: $arr[] = [0, 0]$

Output: $arr[] = [0, 0]$

Explanation: No change in array as there are all 0s.

[Better Approach] Two Traversals – $O(n)$ Time and $O(1)$ Space

Q.8) Given a 1-based indexing array $arr[]$ of **non-negative** integers and an integer **sum**. You mainly need to return the left and right indexes(**1-based indexing**) of that subarray. In case of multiple subarrays, return the subarray indexes which come first on moving from left to right. If no such subarray exists return an array consisting of element **-1**.

Examples:

Input: $arr[] = [15, 2, 4, 8, 9, 5, 10, 23]$, $target = 23$

Output: $[2, 5]$

Explanation: Sum of subarray $arr[2...5]$ is $2 + 4 + 8 + 9 = 23$.

Input: $arr[] = [1, 10, 4, 0, 3, 5]$, $target = 7$

Output: $[3, 5]$

Explanation: Sum of subarray $arr[3...5]$ is $4 + 0 + 3 = 7$.

Input: $arr[] = [1, 4]$, $target = 0$

Output: $[-1]$

Explanation: There is no subarray with 0 sum.

Q.9) Rearrange an array in maximum minimum form using Two Pointer Technique

Given a sorted array of positive integers, rearrange the array alternately i.e first element should be a maximum value, at second position minimum value, at third position second max, at fourth position second min, and so on.

Examples:

Input: $arr[] = \{1, 2, 3, 4, 5, 6, 7\}$

Output: $arr[] = \{7, 1, 6, 2, 5, 3, 4\}$

Explanation: First 7 is the max value, then 1 is the min value, then 6 is the second max value, then 2 is the second min value, then 5 is the third max value, then 3 is the third min value and at last 4 is the fourth max value.

Input: $arr[] = \{1, 2, 3, 4, 5$

$, 6\}$

Output: $arr[] = \{6, 1, 5, 2, 4, 3\}$

Q.10) Accept a number from user and check if it's palindrome number or not.

Q.11) Accept a word from user and check if it is palindrome or not.

Q12) Check Palindrome by Filtering Non-Letters

Given a string containing letters, digits, and symbols, determine if it reads the same forwards and backwards when considering only alphabetic characters (case-insensitive).

Example

Input

code = A1b2B!a

Output

1

Explanation

- Step 1: Extract only letters → ['A','b','B','a']
- Step 2: Convert to lowercase → ['a','b','b','a']
- Step 3: Compare sequence forward and backward: 'abba' == 'abba' → true

Input Format

- A string *code* containing letters (A-Z, a-z), digits (0-9), and symbols

Constraints

- $0 \leq \text{code.length} \leq 1000$
- For all $0 \leq i < \text{code.length}$: $33 \leq \text{ASCII}(\text{code}[i]) \leq 126$
- *code* contains only printable ASCII characters (letters, digits, symbols)

Output Format

- Return a boolean value: 1 if true & 0 if false.

Sample Input 0

Z

Sample Output 0

1

Sample Input 1

abc123cba

Sample Output 1

1

Q.13) Find First Occurrence

Given a **sorted** array of integers that may contain duplicates, return the index of the **first occurrence** of a target value or -1 if not found.

Example

Input:

nums = [1, 2, 3, 4, 5]

target = 3

Output:

2

Explanation:

We perform binary search on [1,2,3,4,5].

1. low=0, high=4 → mid=2 → nums[2]=3 equals target. Record result=2, then search left half.
2. Update high=mid-1=1. Now low=0, high=1 → mid=0 → nums[0]=1 < target, so move low to mid+1=1.
3. low=1, high=1 → mid=1 → nums[1]=2 < target, so move low to mid+1=2.

Now low(2)>high(1), terminate. The first occurrence found is at index 2.

Input Format

The input consists of two lines.

First line: two space-separated integers n and target, where $0 \leq n \leq 1000$ and $-10^9 \leq \text{target} \leq 10^9$.

Second line: n space-separated integers $\text{nums}[i]$, each satisfying $-10^9 \leq \text{nums}[i] \leq 10^9$, and nums is sorted in non-decreasing order.

Edge cases include: $n = 0$ (empty array), all elements less than target, all elements greater than target, multiple duplicates of target, single-element array.

Constraints

- $0 \leq \text{nums.length} \leq 1000$
- $-10^9 \leq \text{nums}[i] \leq 10^9$ for all $0 \leq i < \text{nums.length}$
- $-10^9 \leq \text{target} \leq 10^9$
- For all $0 \leq i < \text{nums.length} - 1$, $\text{nums}[i] \leq \text{nums}[i+1]$ (array is non-decreasingly sorted)

Output Format

Output a single integer: the index (0-based) of the first occurrence of target in the array *nums*.
If target does not exist in *nums*, output -1.

Sample Input 0

0

5

Sample Output 0

-1

Sample Input 1

1

3

3

Sample Output 1

0

Q.14) Maximum Number of Non-Overlapping Intervals

Given an array of intervals where each interval has a start and end time, return the maximum number of non-overlapping intervals.

Examples

Example 1

Input:

meetings = [[1, 2], [2, 3], [3, 4], [1, 3]]

Output:

3

Explanation:

- Step 1: Sort meetings by end time → [[1,2],[2,3],[1,3],[3,4]].
- Step 2: Select [1,2] (count=1, last_end=2).
- Step 3: [2,3] has start 2 ≥ 2, select (count=2, last_end=3).
- Step 4: [1,3] start 1 < 3, skip.
- Step 5: [3,4] start 3 ≥ 3, select (count=3). Result = 3.

Example 2

Input:

meetings = [[0, 5], [0, 1], [1, 2], [2, 3], [3, 5], [4, 6]]

Output:

4

Explanation:

- Step 1: Sort by end time $\rightarrow [[0,1],[1,2],[2,3],[3,5],[0,5],[4,6]]$.
- Step 2: Select $[0,1]$ (count=1, end=1).
- Step 3: $[1,2]$ (count=2, end=2).
- Step 4: $[2,3]$ (count=3, end=3).
- Step 5: $[3,5]$ (count=4, end=5). Skip $[0,5]$ and $[4,6]$ because their start times are before last_end=5. Result = 4.

Input Format

- The first line contains a single integer, meetings_rows, representing the total number of meetings.
- The second line contains a single integer, meetings_columns, representing the number of properties for each meeting (typically 2 for a start and end time).
- The next meetings_rows lines describe the meetings. Each line contains meetings_columns space-separated integers, representing the properties of a single meeting.

6

2

1 2

3 4

0 6

5 7

8 9

5 9

Explanation:

- The first line, 6, is the value for meetings_rows.
- The second line, 2, is the value for meetings_columns.
- The following 6 lines represent the individual meetings, which are parsed into a 2D list: $[[1, 2], [3, 4], [0, 6], [5, 7], [8, 9], [5, 9]]$.

Constraints

- $0 \leq \text{meetings.length} \leq 1000$

- `meetings[i].length == 2` for all $0 \leq i < \text{meetings.length}$
- $0 \leq \text{meetings}[i][0] < \text{meetings}[i][1] \leq 10^9$
- All start and end times are integers
- Meetings may share start or end times

Output Format

- Return a single integer denoting the maximum number of non-overlapping meetings that can be scheduled.

Sample Input 0

1
2
5 10

Sample Output 0

1

Sample Input 1

3
2
1 2
2 3
3 4

Sample Output 1

3



Q.15) Validate Properly Nested Brackets

Given a string, check if all brackets ('()', '{}', '[]') are properly matched and nested. Return 1 if valid, otherwise return 0.

Example

Input

```
code_snippet = if (a[0] > b[1]) { doSomething(); }
```

Output

1

Explanation

All brackets are properly matched: '(' with ')', '[' with ']', and '{' with '}'. No mismatches or improper nesting.

Input Format

- The function takes a single parameter, `code_snippet`, which is a `STRING`.

Constraints

- $0 \leq \text{code_snippet.length} \leq 1000$
- `code_snippet` consists of printable ASCII characters (character codes 32 to 126 inclusive)
- `code_snippet` may contain any combination of '(', ')', '{', '}', '[', ']', letters, digits, symbols, and whitespace
- `code_snippet` may be empty

Output Format

- The function returns a `BOOLEAN` value, 1 for True and 0 for False.

Sample Input 0

```
int x = 42; // no brackets here
```

Sample Output 0

1

Sample Input 1

```
() {} []
```

Sample Output 1

1

Q.16) Given an array of positive integers and a target integer, return the indices of two elements that sum to the target or `[-1, -1]` if no such pair exists.

Example 1

Input:

```
taskDurations = [2, 7, 11, 15]
```

```
slotLength = 9
```

Output:

```
[0, 1]
```

Explanation:

We look for two durations that sum to the slotLength. Starting with index 0 (2), we need 7 (9-2). At index 1, duration is 7. Thus indices [0, 1] sum to 9.

Example 2

Input:

taskDurations = [1, 2, 3, 4]

slotLength = 8

Output:

[-1, -1]

Explanation:

No two durations in the list sum to 8. All pairs are: 1+2=3, 1+3=4, 1+4=5, 2+3=5, 2+4=6, 3+4=7. None match 8.

Input Format

The input consists of three lines:

- An integer n , the number of tasks ($0 \leq n \leq 1000$).
- n space-separated positive integers: `taskDurations[0]`, `taskDurations[1]`, ..., `taskDurations[n-1]` (each between 1 and 1,000,000).
- An integer `slotLength`, the desired sum of two task durations ($1 \leq \text{slotLength} \leq 2,000,000$).

Constraints

- Let $n = \text{taskDurations.length}$
- $0 \leq n \leq 1000$
- $1 \leq \text{taskDurations}[i] \leq 1000000$ for all $0 \leq i < n$
- $1 \leq \text{slotLength} \leq 2000000$
- Indices are 0-based
- If a solution exists, return two distinct indices i, j such that $0 \leq i < j < n$ and $\text{taskDurations}[i] + \text{taskDurations}[j] = \text{slotLength}$
- If no valid pair exists, return `[-1, -1]`

Output Format

Print two space-separated integers i and j ($0 \leq i < j < n$) representing the indices of the two tasks whose durations sum exactly to `slotLength`. If multiple valid pairs exist, any one may be printed. If no valid pair exists, print `"-1 -1"`.

Sample Input 0

0

10

Sample Output 0

-1

-1

Sample Input 1

1

5

5

Sample Output 1

-1

-1

Q.17) Accept a number and print factorial of a number using recursive function.

Q.18)

1 121 12321	1 121 12121	1 1 2 3 3
***** **** *** ** *	1 2 3 5 8 13 [Fibonacci series]	2 5 7 11 13 17 [Prime number]

Q.19) Accept a sentence, accept a word and count occurrence of that word

Q.20) Accept data in 3*3 matrix and transpose it.

Q.21) Write a code for anagram string. An anagram is a word or phrase formed by rearranging the letters of another, using all the original letters exactly once

Q.22) Accept a sentence and reverse each word.

Q.23) Accept a sentence and print all palindrome string within it.

Eg. madam love's coding but nitin hate it. O/P → madam, nitin count=2

Q.24) Max Unique Substring Length in a Session

Given a string of lowercase letters with sessions separated by " characters, find the maximum length of a substring with all distinct letters within any single session.

Example

Input

sessionString = abcabcbb

Output

3

Explanation

- There is only one session: "abcabcbb".
- Scanning with a sliding window for distinct letters, the longest substrings without repeats are "abc", "bca" and so on, each of length 3.
- Therefore, the result is 3.

Input Format

- A single line containing the string sessionString.

Constraints

- $0 \leq S.length \leq 100000$
- For all i in $[0, S.length]$: $S[i]$ is either '*' or a lowercase letter 'a' to 'z'
- Each session is defined as a maximal contiguous substring of S without '*' characters
- Number of sessions (segments between '*') is at most $S.length + 1$

Output Format

- A single integer denoting the maximum length among all substrings. If sessionString is empty or contains only '*', output 0.

Sample Input 0

*

Sample Output 0

0

Sample Input 1

aa

Sample Output 1

1

